This report summarizes the implementation and analysis of three distinct computational tasks: **High-Dimensional Clustering**, **Web-Search Data Structures**, and **PageRank Centrality**.

https://github.com/SuvenduMullick2023/Big_data/tree/main/assignment4

Part 1: High-Dimensional Clustering (Spambase Dataset)

Understanding & Methodology

The goal was to partition the UCI Spambase dataset into clusters to identify patterns in spam vs. non-spam attributes. The dataset consists of 4,601 points with 58 dimensions, representing word and character frequencies.

Reasoning & Assumptions

- **Dimensionality:** With 58 features, the Euclidean distance starts to lose contrast (the "curse of dimensionality"). Standard k-means can be sensitive to this.
- **Normalization:** Since global values range from 0.0 to 15841.0, it is assumed that feature scaling is necessary to prevent high-magnitude features (like total capital letters) from dominating the clustering objective.
- **Metric Comparison:** We compared standard k-means++ with a "Coreset" approach (k-center as a precursor).

Output Analysis

Algorithm	Configuration	Objective (Cost)	Execution Time
k-means++	k=10	**31,249.25**	N/A
k-center → k-means++	k=50 → 10	**420,864.87**	16.98 sec

- **Result Interpretation:** Standard k-means++ achieved a significantly lower objective (31,249$) compared to the two-step k-center approach (420,864). This suggests that for this specific dataset size, direct k-means++ initialization provides a much more optimal clustering than reducing the data to 50 centers first.

Part 2: Web Search Data Structures (Inverted Index)

Understanding & Methodology

The task involved simulating a basic search engine index. Pages are added to the system, and queries find pages containing specific words or the positions of those words.

Reasoning & Assumptions

- **Data Structure:** An **Inverted Index** (likely a Hash Map where keys are words and values are lists of page IDs) is assumed to ensure O(1) or O(\log n) lookup times.
- **Case Sensitivity:** The search for "C" and "C++" returned results for stack_cprogramming, suggesting the index is case-sensitive and handles special characters.

Output Analysis

- **Keyword Association:** Multi-word queries like "function" successfully identified commonality across three distinct pages (stack_cprogramming, stack_datastructure_wiki, stackoverflow).
- **Dynamic Updates:** The system successfully updated its "knowledge" in real-time. For example, "allain" returned no results until stack_cprogramming was added, after which it correctly pointed to that page.
- **Precision:** The query for "magazines" correctly filtered out general "stack" pages and identified only stackmagazine.

Part 3: PageRank Implementation (Graph Centrality)

Understanding & Methodology

PageRank was implemented to measure the relative importance of nodes in two different web graphs: small.txt (100 nodes) and whole.txt (1000 nodes).

Reasoning & Assumptions

- **Damping Factor:** It is assumed a standard damping factor (alpha = 0.85) was used to handle "rank sinks" or nodes with no out-links.
- **Convergence:** The execution on whole.txt involved Spark Stages, implying a distributed iterative approach to handle the 1024+ edges efficiently.

Graph	Scale	Top Node (Score)	Bottom Node (Score)
small.txt	100 Nodes	**Node 53** (0.0357)	**Node 85** (0.0034)
whole.txt	1000 Nodes	**Node 263** (0.0020)	**Node 558** (0.0003)

Score Distribution: In small.txt, the top node holds ~3.5% of the total network "importance," while the bottom node holds only ~0.3%.

Scalability: As the graph grew 10x (from 100 to 1000 nodes), the individual scores dropped proportionally (the top score dropped from 0.035 to 0.002). This confirms that the PageRank scores are correctly normalized (Sum = 1.0) across the entire graph.

Centrality Insight: Nodes like 53 and 263 act as primary hubs or authorities within their respective networks.

Final Summary :

The project demonstrates that while **distributed algorithms** (PageRank/Clustering) are essential for scale, the **initialization strategy** (as seen in Part 1) and **data organization** (Inverted Index in Part 2) are the primary drivers of result quality. The PageRank results specifically confirm that even in larger graphs, a small subset of "hub" nodes consistently dominates the network influence.